第4回課題 解答

学籍番号：25G1017　　氏名：岩澤大心

作成日：2026年10月12日

課題1：$x^2 = 2$をNewton法で解く

$f(x) = x^2 - 2$，$f'(x) = 2x$とおくと，更新式は

$$x_{k+1} = x_k - \frac{x_k^2 - 2}{2x_k}, \qquad x_0 = 2$$

となる．一般のベクトルの場合は，ヤコビ行列Jについて$J(\boldsymbol{x}_k)\boldsymbol{\Delta}_k = F(\boldsymbol{x}_k)$を解き，$\boldsymbol{x}_{k+1} = \boldsymbol{x}_k - \boldsymbol{\Delta}_k$とする． 空欄は`linalg.solve(Df(x), f(x))`で補い，その後に`x = x - DFinv_Fx`を実行する． 逆行列を明示的に作らず，連立一次方程式を直接解く．

```
import numpy as np
from scipy import linalg


def newton(x, f, Df, cond, history=None, max_iter=100):
    """Solve J(x) * step = f(x), then update x <- x - step."""
    x = np.array(x, dtype=float, copy=True)
    with np.errstate(over="raise", invalid="raise", divide="raise"):
        for i in range(max_iter):
            DFinv_Fx = linalg.solve(Df(x), f(x))
            x = x - DFinv_Fx
            if not np.all(np.isfinite(x)):
                raise FloatingPointError("Non-finite Newton iterate")
            if history is not None:
                history.append((x.copy(), DFinv_Fx.copy()))
            if cond(i, x, DFinv_Fx):
                return x
    raise RuntimeError("Newton method did not converge within max_iter")


def f1(x):
    return np.array([x[0] ** 2 - 2.0])


def Df1(x):
    return np.array([[2.0 * x[0]]])


def cond_fixed(i, x, DFinv_Fx):
    # Original stopping rule: indices 0 through 20 mean 21 updates.
    return i == 20
```

```
x = newton([2.0], f1, Df1, cond_fixed)
print(x)  # [1.41421356]
```

計算結果は$x \simeq 1.414213562373095$であり，正の解$\sqrt{2}$に一致する． 更新後に終了判定を行うため，$i = 0, \ldots, 20$の`i == 20`は 計21回の更新に相当する．残差$|x^2 - 2|$は約$4.44 \times 10^{-16}$であった． 配布コードの1要素配列は`x[0]`として参照する． `history`は履歴保存用，`max_iter`は未収束時の無限ループ防止用である．

課題2：更新量による終了条件

更新量の全成分の絶対値を取り，最大値が10^{-14}以下になったら終了する． すなわち，

$$\|\boldsymbol{\Delta}_k\|_\infty = \max_j |(\boldsymbol{\Delta}_k)_j| \le 10^{-14}$$

を判定する．課題1の回数による判定を，次の`cond`に置き換える．

```
def cond(i, x, DFinv_Fx):
    return np.max(np.abs(DFinv_Fx)) <= 10 ** (-14)
```

```
history = []
x = newton([2.0], f1, Df1, cond, history=history)
print(x)
print(len(history))
```

出力は`[1.41421356]`，更新回数は6回となった． 更新後の値と，その更新に用いた修正量の絶対値を表に示す．

更新回数	更新後のx	$\lvert\Delta\rvert$
1	1.500000000000000	$5.000000000 \times 10^{-1}$
2	1.416666666666667	$8.333333333 \times 10^{-2}$
3	1.414215686274510	$2.450980392 \times 10^{-3}$
4	1.414213562374690	$2.123899820 \times 10^{-6}$
5	1.414213562373095	$1.594742910 \times 10^{-12}$
6	1.414213562373095	$1.570092459 \times 10^{-16}$

5回目の更新量は10^{-14}より大きく，6回目で条件を満たす． 課題1と同じ精度の解を，固定回数より少ない更新で得られた．

結果の確認

$$x \simeq 1.414213562373095, \qquad |x^2 - 2| \simeq 4.44 \times 10^{-16}.$$

終了条件は`f(x)`の値ではなく，指定された`DFinv_Fx`の最大絶対値に対する条件である． 残差は解の確認用に別途計算した．例えば，修正量が$(5 \times 10^{-15}, -2 \times 10^{-14})$なら最大絶対値は$2 \times 10^{-14}$であり，終了しない．

課題3：非線形連立方程式

与えられた方程式を

$$F(x,y)=\begin{pmatrix}2x^2-e^y\\ -3xy^2-y^3-3\end{pmatrix}=\mathbf{0}$$

とおく．各成分をx,yで偏微分したヤコビ行列は

$$J(x,y)=\begin{pmatrix}4x & -e^y\\ -3y^2 & -6xy-3y^2\end{pmatrix}$$

である．課題1の`newton`と，課題2の`cond`をそのまま利用する．

```
def f3(p):
    x, y = p
    return np.array([2 * x ** 2 - np.exp(y),
                     -3 * x * y ** 2 - y ** 3 - 3])


def Df3(p):
    x, y = p
    return np.array([[4 * x, -np.exp(y)],
                     [-3 * y ** 2, -6 * x * y - 3 * y ** 2]])
```

```
x = np.zeros(2)
x[0] = 4.0
x[1] = 5.0
history = []
x = newton(x, f3, Df3, cond, history=history)
print(x)
print(len(history))
print(np.max(np.abs(f3(x))))
```

計算結果

指定された初期値$(x_0,y_0)=(4,5)$から，40回の更新で

$$\boxed{x\simeq -1.214687810300159,\qquad y\simeq 1.082121375602253}$$

を得た．小数点以下8桁に丸めると $(x,y)=(-1.21468781,\ 1.08212138)$となり，課題文の近似解と一致する．

最終更新量の最大絶対値は約1.18×10^{-16}，残差は

$$\|F(x,y)\|_\infty\simeq 4.44\times10^{-16}$$

であった．以上より，終了条件を満たし，元の2式も十分な精度で成立している． この初期値では20回固定の更新では収束しないため， 課題2の更新量による終了条件を用いた．

課題4：初期値を変えて別の解を探す

課題3と同じ関数・ヤコビ行列・終了条件を使い，初期値を変更した． 課題3以外に得られた近似解は，次の2種類である．

$$\text{解A：}\quad (x, y) \simeq (0.288166073001912,\ -1.795289464785586),$$
$$\text{解B：}\quad (x, y) \simeq (-0.401282369883953,\ -1.133032689807632).$$

これらは課題3の$y > 0$の解とは異なり，いずれも$y < 0$である．

```
for initial in ([1.0, -2.0], [4.0, -5.0], [1.0, 1.0],
                [-1.0, -1.0], [-4.0, -5.0], [2.0, -2.0]):
    history = []
    root = newton(initial, f3, Df3, cond, history=history)
    print(initial, root, len(history),
          np.max(np.abs(f3(root))))
```

初期値(x_0, y_0)	収束先	更新回数	$\|F(x, y)\|_\infty$
$(1, -2)$	解A	10	4.44×10^{-16}
$(4, -5)$	解A	9	4.44×10^{-16}
$(1, 1)$	解A	12	4.44×10^{-16}
$(-1, -1)$	解B	7	4.44×10^{-16}
$(-4, -5)$	解B	10	8.88×10^{-16}
$(2, -2)$	解B	12	8.88×10^{-16}

確認と考察

課題3の解と合わせて，今回の計算では3種類の解を確認した． 例えば$(1, -2)$と$(2, -2)$は，初期値の符号が同じでも異なる解へ収束した． Newton法の収束先は初期値に依存し，初期値の符号だけでは決まらない． また，特異なヤコビ行列や発散が生じる初期値もあるため， 終了条件の成立に加えて残差も確認した．

第2式から$y = 0$は不可能であり，$y \neq 0$では

$$x = -\frac{y}{3} - \frac{1}{y^2}$$

と整理できる．得られた3種類の解は，この関係と第1式 $2x^2 = e^y$の両方を数値的に満たす．

再実行方法

同梱の`assignment4.py`を`python assignment4.py`で実行すると， 課題1–4の初期値・近似解・更新回数・最終更新量・残差を表示する． 実行環境はPython 3，NumPy 2.3.5，SciPy 1.17.0である． 表の残差は丸め表示前の計算値から求めた．